Arquitectura basada en representación intermedia para la generación confiable de consultas SQL/PGQ desde lenguaje natural

Plan de tesis – Avance parcial

Guido Luis Tapia Oré
Asesora: Dra. Regina Paola Ticona Herrera

Universidad Católica San Pablo
Escuela Profesional de Ciencia de la Computación

2026

Contenido

Motivación y problema

Objetivos

Estado del arte

Propuesta

Validación

Estado y siguientes pasos

Motivación: Cresciente interés del mercado con una respuesta poco estructurada

Pregunta de negocio (NL) → Sistema *text-to-query* → Resultado sobre la BD

Demanda empresarial
Organizaciones *data-driven*
Self-service analytics
Decisiones en ciclos cortos

LLM como motor
Adopción acelerada
Generalización a esquemas
Lenguaje natural como interfaz primaria

Oferta actual: ingenua
Esquema inyectado al *prompt*
Auto-corrección textual
Sin verificación estructural

Motivación: el esquema híbrido SQL/PGQ como oportunidad

Relacional dominante
Madurez, integridad, estabilidad de producción

Grafos en alza
Modelado natural de rutas y vecindarios: fraude, ciberseguridad, *supply chain*

Estandarización
ISO/IEC 9075-16:2023 (SQL/PGQ)
Grafos consultados desde SQL

Vacío en text-to-query
SOTA cubre solo el lado relacional; el régimen híbrido queda sin abordar

Oportunidad de investigación

El estándar híbrido *ya existe* y *ya tiene motores que lo implementan* —DuckPGQ (Wolde et al., 2023) sobre ISO/IEC 9075-16:2023 (ISO/IEC JTC 1/SC 32, 2023). Lo que falta es una arquitectura *text-to-query* que opere de forma confiable sobre este régimen.

El problema: Carencia de técnicas text-to-query sobre un esquema dual SQL/PGQ

Esquema dual SQL/PGQ

Esquema relacional
tablas, columnas, claves, tipos

Grafo de propiedad
vértices, aristas, etiquetas, propiedades

Estandarizado en ISO/IEC 9075-16:2023 (ISO/IEC JTC 1/SC 32, 2023), soportado por DuckPGQ (Wolde et al., 2023).

Modos de fallo del LLM:

- Tablas o columnas inexistentes.
- Rutas de *join* inventadas.
- Etiquetas de vértice o arista inventadas.
- Patrones de grafo mal formados.
- **Incoherencia cruzada** relacional–grafo (categoría nueva).

Objetivos

Objetivo general. Proponer una arquitectura de razonamiento estructural basada en representación intermedia (RI) para la generación *text-to-query* sobre SQL/PGQ.

Objetivos específicos.

O1. Diseñar IR-SQL/PGQ con tipado y compilación determinista

O2. Implementar pipeline con anclaje, generación estructurada y verificación estática+dinámica

O3. Evaluar con métricas canónicas (EX, EM, VES)

O4. Caracterizar transferibilidad y coste agregado por consulta (R7, R8)

Estado del arte y brecha

Línea	Aporte principal	Limitación frente a SQL/PGQ
Decodificación restringida (Scholak et al., 2021)	Validez sintáctica por *token*	Nivel léxico; no captura coherencia cruzada
Descomposición / agentes (Pourreza & Rafiei, 2023; Wang et al., 2024)	Genera por sub-tareas	Solo régimen relacional puro
Anclaje pre-generación (Qu et al., 2024)	Reduce alucinaciones por *grounding*	No contempla esquemas duales
Schema linking y M-Schema (Cao et al., 2024; Gao et al., 2024)	Representación rica del esquema	Solo bloque relacional
Verificación por ejecución (Talaei et al., 2024)	Bucle correctivo desde el motor	Retroalimentación textual frágil
Razonamiento tabular–grafo (Stoisser, Martell et al., 2025)	Unifica paradigmas con RL	No produce SQL/PGQ estándar

Brecha identificada

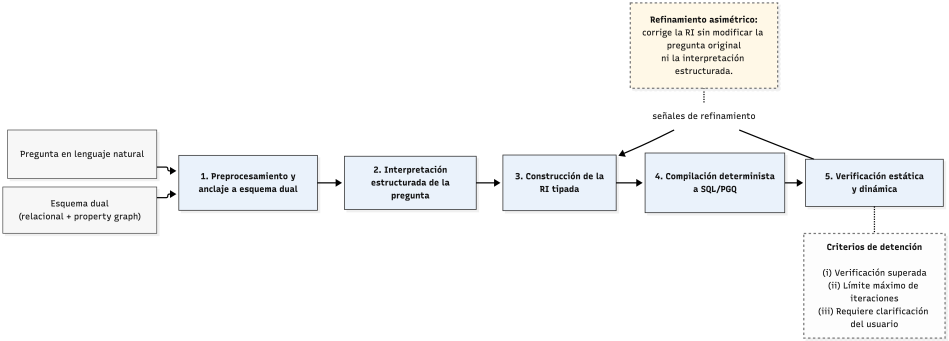
Mecanismos maduros *pero* fragmentados y centrados en SQL relacional.
Falta una arquitectura que coordine **anclaje + RI tipada + verificación** sobre el estándar híbrido SQL/PGQ.

Desempeño actual: el SOTA decae al salir del laboratorio

Sistema	**Spider**	**BIRD**	**Spider 2.0**	**Text2GQL**
	EX (lab clásico)	*EX (intermedio)*	*EX (empresarial)*	*EX (ISO-GQL)*
LLMs *zero-shot* (Lyu et al., 2026)	~ 72	~ 35	~ 12	≤ 4
DIN-SQL (Pourreza & Rafiei, 2023)	85	50	6	—
MAC-SQL (Wang et al., 2024)	86	60	?	—
CHESS (Talaei et al., 2024)	87	68	17	—
XiYan-SQL (Gao et al., 2024)	89	73	12	—
Qwen3-8B *fine-tuned* (8B abierto)	—	—	—	45

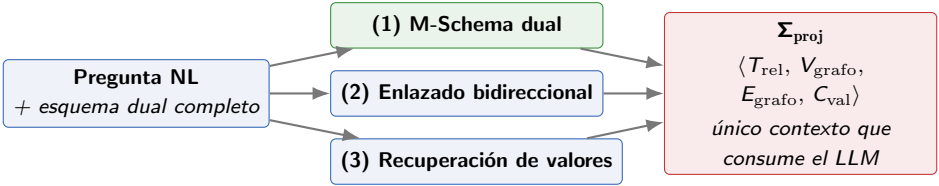
Valores aproximados de Execution Accuracy (EX, en porcentaje) reportados por los autores en sus publicaciones de referencia.

Pipeline propuesto



- **(1) Anclaje al esquema dual.** Construye $\Sigma_{\text{proj}} = \langle T_{\text{rel}}, V_{\text{grafo}}, E_{\text{grafo}}, C_{\text{val}} \rangle$.
- **(2) Interpretación estructurada** de la pregunta.
- **(3) Construcción de la RI tipada** IR-SQL/PGQ.
- **(4) Compilación determinista** a SQL/PGQ.
- **(5) Verificación** estática y dinámica con bucle de refinamiento sobre la RI.

Anclaje al esquema dual: el esquema proyectado Σ_{proj}

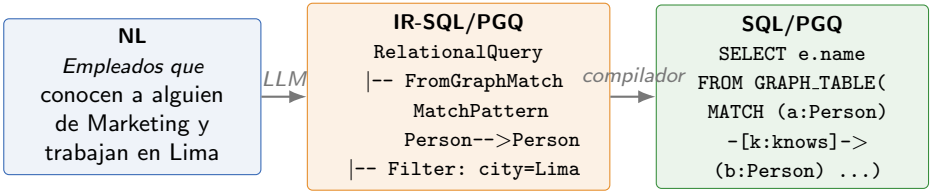


Procedencia y estado

M-Schema de XiYan-SQL (Gao et al., 2024) extendido con descriptores de grafo, **enlazado bidireccional** de RSL-SQL (Cao et al., 2024) y **recuperación** estilo CHESS (Talaei et al., 2024). Hoy opera (1) sobre la estructura completa; (2) y (3) se incorporarán para esquemas masivos como BIRD donde el filtrado por relevancia es indispensable.

IR-SQL/PGQ: vista intuitiva

Ejemplo. *¿Qué empleados conocen a alguien del equipo de Marketing y trabajan en Lima?*



Idea central

La RI es el **contrato fijo** entre el modelo y el resto del pipeline. El LLM *nunca* emite un query libre: emite IR validada por tipos, que un compilador determinista traduce a SQL/PGQ.

IR-SQL/PGQ: árbol y tres bloques cerrados

Bloque relacional
`TableRef`, `ColumnRef`
Expresiones recursivas
`RelationalQuery`

Bloque de grafo
`VertexPattern`
`EdgePattern`, `PathPattern`
`MatchPattern`

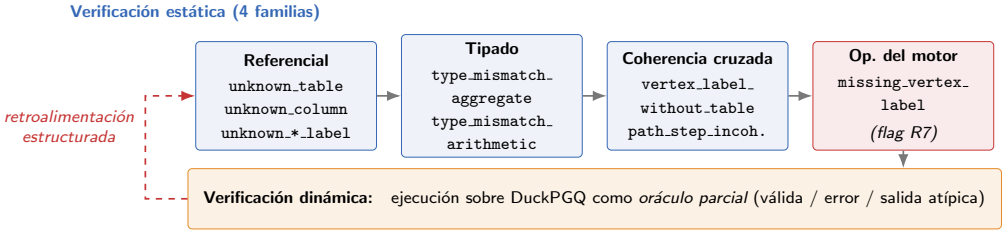
Composición híbrida
`FromGraphMatch`
Encaja en `FromExpr`
Tres ámbitos separados

Cualquier consulta del alcance es composición de estos tres bloques

Linaje del diseño

- **SemQL** (Guo et al., 2019): descomposición tipada.
- **NatSQL**: uniformidad operacional.
- **RESDSQL** (H. Li et al., 2023): compilador determinista.

Verificación en cascada



Política de retroalimentación

El bucle no reenvía al LLM el mensaje crudo del motor. Lo traduce a un **descriptor categórico** que se inyecta en la etapa de construcción de la RI: vocabulario común, no oscilaciones por texto libre.

Plan experimental: stack, datasets y régimen

Stack técnico fijado.

- **Motor:** DuckDB 1.4.4 + extensión `duckpgq` (Wolde et al., 2023).
- **LLM:** Claude Haiku 4.5; sustituible bajo P5.
- **Parser SQL:** sqlglot, dialecto `duckdb`.
- **Entorno:** `uv` 0.11+, Python 3.11.
- **Marco evaluación canónica:** NL2SQL360 (B. Li et al., 2024).

Datasets por régimen

Régimen	Dataset
Relacional puro	Spider (Yu et al., 2018), BIRD (J. Li et al., 2023), Spider 2.0 (Lei et al., 2025)
Grafo puro (Cypher / ISO-GQL)	Text2GQL-Bench (Lyu et al., 2026)
Híbrido SQL/PGQ	Corpus derivado

El corpus híbrido se construye *siguiendo la metodología* de Text2GQL-Bench adaptada a SQL/PGQ.

Comparación contra *baselines*

Baseline	Anclaje	RI tipada	Verif. cascada	Cobertura SQL/PGQ
DIN-SQL (Pourreza & Rafiei, 2023)	parcial	no	no	no
MAC-SQL (Wang et al., 2024)	parcial	no	ejecución	no
CHESS (Talaei et al., 2024)	sí	no	ejecución	no
RSL-SQL (Cao et al., 2024)	sí	no	parcial	no
XiYan-SQL (Gao et al., 2024)	sí	implícita (M-Schema)	ejecución	no
STRuCT-LLM (Stoisser, Martell et al., 2025)	parcial	no	no	tabular+grafo, no estándar
Esta tesis	**dual**	**sí (IR-SQL/PGQ)**	**estática+dinámica**	**sí (estándar ISO)**

Métricas, ablations y avance actual

Métricas reportadas.

- **Canónicas:** EX, EM, VES (B. Li et al., 2024).
- **Híbrido SQL/PGQ:** alucinación referencial, coherencia cruzada, descriptores categóricos.
- **Coste por consulta (R8):** invocaciones LLM, *tokens* in/out, iteraciones del bucle, latencia P50/P95, coste monetario.

Ablations.

- Con/sin RI tipada.
- Con/sin verificación dinámica.
- Con/sin coherencia cruzada.

Avance actual.

- ✓ Dataclasses de IR-SQL/PGQ y JSON Schema.
- ✓ Compilador determinista IR → SQL/PGQ.
- ✓ Verificador estático: 4 familias.
- ✓ *Lifter* SQL → IR: 99 % *round-trip*.
- → Bucle de retroalimentación con clasificador de errores.
- ☐ Adaptación del corpus a SQL/PGQ.
- ☐ Evaluación canónica con NL2SQL360.
- ☐ *Ablations* sistemáticos.

Estado y siguientes pasos

Estado actual.

- Brecha confirmada: las técnicas de mitigación de alucinaciones están fragmentadas y no cubren el régimen híbrido SQL/PGQ (Cuadro de *baselines*).
- La mediación por RI tipada traslada al espacio de la representación errores que en la práctica común se detectan tarde o se ocultan tras retroalimentación textual.
- Prototipo funcional: *lifter*, compilador determinista y verificador estático integrados sobre DuckPGQ.

Siguientes pasos.

- Ampliación de la cobertura de la RI: *window functions* y mecanismos de ambigüedad inspirados en (Ding et al., 2025).
- *Ablations* sistemáticos sobre el bucle de verificación.
- Evaluación con LLMs adicionales bajo P5; reporte de coste agregado por consulta (R8).

Gracias por su atención.

Quedo a su disposición para preguntas y observaciones.

Guido Luis Tapia Oré – UCSP – 2026

Referencias I

Cao, Z., et al. (2024). RSL-SQL: Robust Schema Linking in Text-to-SQL Generation. https://arxiv.org/abs/2411.00073

Ding, Z., Lin, Y., Zeng, T., et al. (2025). AmbiSQL: Interactive Ambiguity Detection and Resolution for Text-to-SQL. https://arxiv.org/abs/2508.15276

Gao, Y., Liu, Y., Li, X., et al. (2024). XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL. https://arxiv.org/abs/2411.08599

Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., & Zhang, D. (2019). Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. https://arxiv.org/abs/1905.08205

ISO/IEC JTC 1/SC 32. (2023, junio). *Information technology – Database languages – SQL – Part 16: Property Graph Queries (SQL/PGQ)* (Standard). International Organization for Standardization. Geneva, CH. https://www.iso.org/standard/79473.html

Referencias II

Lei, F., Su, J., Chen, Y., et al. (2025). Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. *ICLR.* https://arxiv.org/abs/2411.07763

Li, B., Luo, Y., Chai, C., et al. (2024). The Dawn of Natural Language to SQL: Are We Fully Ready? https://arxiv.org/abs/2406.01265

Li, H., Zhang, J., Li, C., & Chen, H. (2023). RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. *AAAI.* https://arxiv.org/abs/2302.05965

Li, J., Hui, B., Qu, G., et al. (2023). Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *NeurIPS.* https://arxiv.org/abs/2305.03111

Referencias III

Lyu, S., Ban, L., Wu, Z., Luo, T., Liu, J., Ma, C., Luo, Y., Tang, N., Qi, S., Lin, H., Liu, Y., & Hong, C. (2026). Text2GQL-Bench: A Text to Graph Query Language Benchmark [Experiment, Analysis & Benchmark]. https://arxiv.org/abs/2602.11745

Pourreza, M., & Rafiei, D. (2023). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *NeurIPS*. https://arxiv.org/abs/2304.11015

Qu, G., Li, J., Li, B., et al. (2024). Before Generation, Align it! A Novel and Effective Strategy for Mitigating Hallucinations in Text-to-SQL Generation. *ACL Findings*. https://arxiv.org/abs/2405.15307

Scholak, T., et al. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *EMNLP*. https://arxiv.org/abs/2109.05093

Referencias IV

Stoisser, J. L., Martell, M. B., et al. (2025). STRuCT-LLM: Unifying Tabular and Graph Reasoning with Reinforcement Learning for Semantic Parsing. *ICLR*. https://openreview.net/forum?id=xZDoGrMTGI

Talaei, S., Pourreza, M., Chang, Y.-C., et al. (2024). CHESS: Contextual Harnessing for Efficient SQL Synthesis. *ICML*. https://arxiv.org/abs/2405.16755

Wang, B., Ren, C., Yang, J., et al. (2024). MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. *COLING*. https://arxiv.org/abs/2312.11242

Wolde, D. t., Szárnyas, G., & Boncz, P. (2023). DuckPGQ: Bringing SQL/PGQ to DuckDB. *Proc. VLDB Endow.*, *16*(12), 4034-4037.

Yu, T., Zhang, R., Yang, K., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP*. https://arxiv.org/abs/1809.08887